

1. Explain programming and python in detail

Programming - Definition and purpose.

Programming is the process of designing, writing, testing and maintaining a set of instructions that a computer follows to perform specific task. These instructions are written using programming languages such as C, Java, python, etc.

* Purpose of programming :-

- To solve real world problems logically and efficiently.
- To automate repetitive tasks.
- To develop software applications.
- To process and analyse data.
- To control hardware and embedded systems.

* Python - Introduction :-

Python is a high-level interpreted, general-purpose programming language developed by Guido Van Rossum and released in 1991. Python emphasises code readability and simplicity.

* Characteristics of python :-

1. Simple and easy to learn -

Python uses English-like syntax, making it beginner friendly.

2. Interpreted language :-

code is executed line by line, making debugging easier.

3. High-level language :-

The programmer does not need to manage memory manually.

4. Object-oriented :-

Supports classes, objects, inheritance and polymorphism

5. Portable :-

Python programs can run on different platforms

6. Extensive library support :-

Includes libraries for data science, web development, AI, ML, etc

7. Open source :-

Free to use and modify.

* Applications of python :-

Python is widely used in

- Web Development
- Data Science and Analytics
- AI, ML
- Automation and scripting
- Scientific computing
- Game Development
- Cyber Security and Networking.

* Types of comments in python :-

Comments are used to explain code and improve readability. Python ignores comments during execution.

1. Single-line comments :-

used for short explanation [using #]

Syntax :-

This is a single-line comment

2. Multi-line comment :-

used for long explanation [using triple quotes]

Syntax :-

```
'''
```

```
This is multi-line comment
```

```
It can span multi lines
```

```
'''
```

* Importance of python in Modern software Development :-

- python is widely used in AI, ML, Data science, cloud computing
- Reduces development time due to simple syntax.
- Strong community support and continuous updates.
- Used by top companies like google, facebook, Netflix and Amazon.
- Ideal for rapid application development and prototyping.

2. Describe Datatypes and operators in python with suitable examples.

* Datatypes - Introduction :-

In python, datatypes define the type of data a variable can store, while operators are symbols used to perform operations on variables and values. Understanding datatypes and operators is essential for working effective python programs.

* Built in Datatypes in python

python provides several built in datatypes, which are broadly classified as follows :-

1. Numeric Datatypes :-

Numeric datatypes stores Numeric Values

- * int - whole number

- * float - Decimal number

- * complex - Number with real and imaginary parts

Ex :-

a = 10 # int

b = 3.5 # float

c = 2 + 4j # complex

2. Sequence datatypes :-

Sequence datatypes store multiple values in an ordered manner.

- * str - Sequence of characters denoted in " "

- * list - mutable collection expressed in []

- * tuple - Immutable collection expressed in ()

Ex :-

name = "vijaya"

marks = [81, 90, 99]

coordinates = (10, 20)

3. Set datatypes :-

A set stores unique elements and does not maintain order

Ex :-

Sub = {"maths", "physics", "chemistry"}

4. Mapping datatypes :-

mapping stores data in key-value pairs

* dict - Dictionary

Ex :-

student = {"Name": "Vijaya", "Roll-no": 23, "branch": "BCA"}

5. Boolean Datatype :-

Boolean datatypes represents True or false values

Ex :-

is - pass = True.

* Type Identification using type () :-

The type () function is used to identify the datatype of a variable

Ex :- x = 10

y = "Hello"

Print (type (x)) # <class 'int'>

Print (type (y)) # <class 'str'>

* Python operators :-

operators are special symbols used to perform operations on variables.

1. Arithmetic operator :-

used for mathematical operations.

Operator	Name	Description	Example
+	Addition	Adds two values	$10 + 5 = 15$
-	Subtraction	subtract two values	$10 - 5 = 5$
*	Multiplication	Multiplies values	$10 * 5 = 50$
/	division	Returns quotient	$10 / 5 = 2.0$
%	modulus	Returns remainder	$10 / 5 = 0$
**	exponent	power operations	$2 ** 3 = 8$
//	floor division	Quotient without decimal	$10 // 3 = 3$

Real-world usage :-

Arithmetic operators are used in calculators, billing system, banking applications.

2. Assignment operators :-

used to assign values to variables

Operators	Description	Example
=	Assign	$a = 10$
+=	Add and Assign	$a += 5$
-=	Subtract and Assign	$a -= 5$
*=	multiply and Assign	$a *= 5$
/=	divide and Assign	$a /= 5$
%=	modulus and Assign	$a \% = 2$
**=	power and Assign	$a ** = 2$
//=	floor divide and Assign	$a // = 2$

Real world usage :-

Assignment operator is used in updating account balance, counters and scores

3. Comparison [Relational] operators :-

used to compare two values and return True or False

operator	Meaning	Example
$=$	equal to	$a = b$
$\neq$	Not equal to	$a \neq b$
$>$	Greater than	$a > b$
$<$	less than	$a < b$
$\geq$	Greater than or equal	$a \geq b$
$\leq$	less than or equal	$a \leq b$

Real - world usage :-

Eligibility checks, pass / fail conditions, ranking systems

4. logical operations :-

logical operations are used to combine conditional expressions.

operators	Description
and	True if both conditions are true
or	True if at least one condition is true
not	Reverse condition.

ex :-

age = 19

print = (age > 18 and age < 60)

print = (age > 18 or age < 60)

Real - world usage :-

logical operators are used in logical validation, form validation, decision making and login Authentication Systems.

5. membership operators :-

used to test membership in a sequence

operators	meaning
in	value exists
not in	value doesnot exists

Ex:- colors = ["red", "blue", "violet", "pink"]
print = ["red" in colors]

Real - world usage :-

searching data in lists , checking user permissions

6. Identity operator :-

used to compare memory locations returns value either True or False

Operator	Meaning
is	Same object
is not	Different object

Ex:- a = [1, 2, 3]

b = a

print (a is b)

Real - world usage :-

Identify operator is used in memory management
object comparison

3. Explain python input and output operations in detail

Introduction :-

Input and output operations allow a program to interact with the user. Input means taking data from the user and output means displaying results to the user. Python provides simple and powerful functions like `input()` and `print()` to handle I/O operations.

* Input () function :-

The input function is used to take input from the user through keyboard.

Syntax :-

Variable `input ("Message to user")`

Default datatype of `input()` :-

* This `input()` function always returns data in "string" (str) format, regardless of what the user enters.

Ex:-

```
name = input ("enter your name")  
print (name)
```

even if the user enters a number

```
age = input ("enter age:")  
print (type (age)) # <class 'str'>
```

* Type conversion while taking input :-

Since `input()` returns a string, type conversion is required when numerical operations are needed.

- common type conversion functions :-

- * `int()` - converts to integer
- * `float()` - converts to decimal
- * `str()` - converts to string

Ex:- Integer input

```
age = int (input ("enter your age:"))  
print (age + 1)
```

Float input :-

```
sal = float int (input ("enter your salary:"))  
print (sal)
```

Multiple type conversion

```
a = int (input ("enter first number:"))  
b = float (input ("enter second number:"))  
print (a+b)
```

* Taking multiple Inputs:-

python allows taking multiple inputs using `split()`

Method 1 :- using multiple `input()` statements

```
name = input ("enter your name:")  
age = int (input ("enter your age:"))  
branch = input ("enter branch:"))
```

Method 2:- Input as string

```
a, b = input ("enter two values:").split()  
print (a)  
print (b)
```

Multiple inputs with type conversion

```
x, y = map (int, input ("enter two numbers:").split())  
print (x+y)
```

whereas

* `split()` separates input values

* `map()` converts each values to integers

* Output using print() function :-

The print() function is used to display output on the screen.

Syntax:- `print (obj1, obj2, sep = ' ', end = '\n')`

* Formatted output using print()

Formatted output improves readability and presentation of results.

① Using separator [sep] :-

The sep parameter separates multiple values

Ex:-

```
print (10, 20, 30, sep = " ") # 10, 20, 30
```

② Using End parameter [end]

The end parameter changes the ending character

Ex:-

```
print ("Hello", end = " ") # Hello World
print ("World")
```

* The end parameter changes the default newline behaviour.

* Format specifiers in python :-

Format specifiers are used to control the output format

② Using % operator [old style formatting]

```
name = "Vijaya"
```

```
age = 21
```

```
Print = ("Name: %s Age: %d" % (name, age))
```

Specifies	meaning
% d	integer
% f	float
% s	string

⑥ using format () method :-

name = "python"

version = 3.10

Print ["language: {} version: {}".format(name, version)]

⑦ using f-strings (modern formatting)

marks = 95

Print [f "marks obtained : {marks}"]

④ Discuss control statement and Decision making statements in python.

Introduction :-

In python programming, control statements are used to control the flow of execution of a program. Normally, Python executes statements sequentially control statements allows a program to make decisions, Repeat tasks or transfer control to another part of the program based on conditions

* Meaning and Importance of control statements

meaning :-

control statements determine which statement will be executed, how many times it be executed and under what condition it should be executed.

Importance :-

- enable decision making in programs
- Allow repetition of tasks using loops
- improve program efficiency and flexibility
- Helps in writing logical and dynamic programs
- essential for solving real world problems.

* Types of control statements in python

Python control statements are broadly classified into three types.

1. Decision - Making statements

used to execute different blocks of code based on conditions. They are

- * if
- * if - else
- * if - elif - else

2. looping [Iterative] statements
Used to repeat a block of code

- * for loop
- * while loop

3. Jump [control transfer] statements
used to change the flow of execution

- * break
- * continue
- * pass

* Decision making statements in python

Decision making statements evaluates a conditions and execute code based on wheather the condition is True or false

① if statement :-

Definition :- The if statement executes a block of code only when the condition is true.

Syntax :- if condition :
 statements

Ex :- marks = 75
 if marks >= 40
 Print (pass)

Real world usage :- checking eligibility, pass/fail status

② if - else statements :-

Definition :- The if-else statement executes one block if the condition is true and another block if the condition is false.

Syntax :- if condition :
 statement

else :
 statements

ex :- age = 16

if age $\geq$ 18 :

Print ("eligible")

else :

print ("Not eligible")

Real-world use :- login validation , access control

© if - elif - else statements :-

Definition :-

if elif - else statements is used to check multiple conditions

Syntax :-

if condition 1 :

statements

elif condition 2 :

statements

elif condition 3 :

statements

else : statements

Ex :- marks = 85

if marks $\geq$ 90

print ("Grade A")

elif marks $\geq$ 75

Print ("Grade B")

elif marks $\geq$ 50

print ("Grade C")

else :

print ("fail")

Real - world use :-

Grading system , menu driven programs

* Syntax flow and execution control in python

In python, programs execute statements sequentially, that is line by line from top to bottom. Execution flow control refers to how the flow of execution is changed using control statements.

1. Sequential execution flow

- * Statements are executed one after one in the order

- * No conditions or repetitions are involved

Syntax :-
statement 1
statement 2
statement 3

Ex :-
a = 10
b = 20
c = a + b
print (c)

2. Conditional execution flow

The program executes different blocks based on conditions

① if statement execution flow :-

- * condition is evaluated
- * if true, if block executes
- * if false, block is skipped

Ex :-
marks = 40
if marks >= 40:
 print ("pass")

(b) if-else statements flow

- * condition is checked
- * if True, if block runs
- * if false, else block runs

Ex:- num = 7

```
if num % 2 == 0
    print("Even")
```

```
else: print("odd")
```

(c) if-elif-else statement flow

- * Condition is checked
- * if false, condition is checked
- * first true blocks executes
- * if All conditions are false, else statement executes

(3) looping execution flow

loops repeat a block of code until a condition fails

(a) for loop flow:-

Syntax :- for variable in sequence
statements

execution flow:-

- * Takes one value from sequence
- * executes loop body
- * Repeat until sequence ends

Ex:-

```
for i in range(1, 4):
    print(i)
```

⑤ while loop flow:-

Syntax:- while condition
statements

execution flow:-

- * condition is checked
- * if true, loop body executes
- * conditions is rechecked
- * loop stops when condition fails

Ex:- i = 1
while i <= 3
Print (i)
i += 1

④ Control Transfer statement flow

Jump statements changes the normal flow of execution

① break statement :- Terminates the loop immediately

Ex:- for i in range (1, 6):
if i == 4
break
Print (i)

② continue statement :- skips the current iteration

Ex:- for i in range (1, 6):
if i == 3
continue
Print (i)

③ pass statement :-
used as a placeholder

Ex:- if True
Pass

5. write an essay on python programming fundamentals

Introduction :-

Programming plays a crucial role in the modern digital world by enabling humans to communicate with computers and solve real world problems efficiently among various programming languages available today python has emerged as one of the most popular and widely used language due to its simplicity readability efficient and scalable software applications use of comment for code documentation

* Role of programming in problem solving

programming is systematic approach to solving problems by breaking them down into smaller, manageable steps and expressing these steps in a language that a computer can understand. It involves understanding the problem, designing an algorithm, writing code, testing, and refining the solution.

In problem solving, programming helps to :

* Automate repetitive tasks

* process large Amount of data quickly and accurately

* Implement logical thinking and decision making

* Develop efficient and optimized solution

Python supports problems solving by offering high-level constructs, built in functions and libraries that reduce complexity and allow programmer to focus on logic rather than low-level detail.

* Python syntax simplicity and Readability

One of the key reason for python's popularity is its simple and readable syntax. python closely resembles the english language. making it easy for beginners to learn and understand unlike many other programming language. python does not use complex symbols such as braces { } or semicolons ; to define code blocks.

Python was indentation to define blocks of code.
which enforces clean and structured programming practices
This features improves readability and reduces syntax errors.
for ex:-

```
if x > 0:  
    print ("positive numbers")
```

such clarity makes python an ideal language for students, educators and professionals.

* Use of comments for code Documentation

comments are used in python to explain code and make it easier to understand. They are ignored by the python interpreter and do not affect program execution comments serve as a documentation, helping programmers and others understand the purpose and logic of the code.

python supports:

- * single line comments (#)

- * multi line comments (" or " " ")

ex:-

```
# This is single line comment
```

```
" " "  
This is multi line comment  
used for documentation
```

```
" " "
```

comments improve code readability, simplify debugging and make maintenance easier

* Datatypes, operators and input/output operations

Datatypes :-

python provides various built-in datatypes to store different kind of data common data type include

* Numeric : int, float, Complex

* Sequence : string, list, tuple

* Set and dictionary

* Boolean : True and false

* Operators :-

Operators are used to perform operations on data

* Arithmetic operator [+ , - , * , /]

* Comparison operators [= , > , <]

* Logical operators [and , or , not]

* Input and output :-

Input and output operations allow interaction with users

* input() - is used to take user input

* print() is used to display the output

Ex :- a = int(input("enter a:"))

b = int(input("enter b:"))

print("sum: ", a+b)

These features make python suitable for real world applications

* Control flow using Decision making statements.

Control flow refers to the order in which statements in a program are executed. In python, control flow is managed using decision making statements, which allow the program to make choices based on certain conditions. These statements enable the program to behave differently for different inputs making it dynamic and intelligent

Decision making in python is primarily achieved using the if, if-else, if-elif-else statements. These statements evaluate conditions that return either True or False and execute code blocks accordingly.

Common decision making statements

- * if
- * if-else
- * if-elif-else

Ex:- marks = 85

if marks >= 90 :

Print ("Grade A")

elif marks >= 75

Print ("Grade B")

else :

Print ("Grade C")

These statements allow programs to make decisions and behave intelligently.

1. Movie ticket pricing

A movie theatre charges :

150 for children (age < 13)

250 for Adults (age 13-59)

200 for seniors (age > 60)

if the person is watching a 3D movie add 50/- extra
WAP that takes age and is 3D [1 or 0] and prints the final ticket price

```
age = int (input("Enter your age:"))
```

```
is 3D = int (input("enter 1 if you are watching 3D movie  
else enter 0:"))
```

```
if is 3D == 1 :
```

```
    if age < 13 :
```

```
        Print ("Ticket price is 200/-")
```

```
    elif age > 13 and age < 59 :
```

```
        Print ("Ticket price with extra 50 is ₹300")
```

```
    else :
```

```
        Print ("Ticket price with extra 50 is 250/-")
```

```
elif age < 13 :
```

```
    print ("Ticket price is 150")
```

```
elif age > 13 and age < 59 :
```

```
    print ("Ticket price is 250")
```

```
else : Print ("Ticket price is 200")
```

Output:-

enter your age : 25

Enter 1 if you are watching a 3D movie else 0:0

Ticket price is 250/-

2. College Attendance rule

A student is allowed to write the exam if
attendance ≥ 75 or

attendance ≥ 60 and has medical certificate

Take attendance percentage and medical certificate and print
"Allowed" or "Not Allowed"

```
Perc = int (input ("enter Attendance percentage:"))
```

```
med = int (input ("enter 1 if you have medical certificate  
else enter 0:"))
```

```
if perc  $\geq$  75:
```

```
    print ("Allowed for exam")
```

```
elif perc  $\geq$  60 and med == 1:
```

```
    print ("Allowed for exam")
```

```
else:
```

```
    print ("Not Allowed for exam")
```

Output :-

Enter attendance percentage : 65

enter 1 if you have medical certificate else enter 0: 1

allowed for exam

③ E-commerce different discount

A shopping list gives:

20% discount if bill ≥ 5000

10% discount if bill is between 2000 and 4999

No discount if bill < 2000

But if the customer is a prime member they get extra 5% discount.

Print final Amount to be paid.

```
bill = int(input("enter bill Amount:"))
```

```
isprime = int(input("enter 1 if you are prime member else  
enter 0:"))
```

```
if isprime == 1:
```

```
    if bill  $\geq$  5000:
```

```
        bill -= bill * 0.25
```

```
    print ("bill after discount:", bill)
```

```
elif bill  $\geq$  2000 and bill  $\leq$  4999:
```

```
    bill -= bill * 0.15
```

```
    print ("bill after discount:", bill)
```

```
else: print ("No discount:", bill)
```

```
elif bill  $\geq$  5000:
```

```
    bill -= bill * 0.2
```

```
    print ("bill after discount:", bill)
```

```
elif bill  $\geq$  2000 and bill  $\leq$  4999:
```

```
    bill -= bill * 0.1
```

```
    print ("bill After discount:", bill)
```

```
else: print ("No discount:", bill)
```

Output:-

Enter bill Amount : 4500

enter 1 if you are prime member else enter 0: 0

bill after discount : 4050

4 Smartphone battery warning

A phone shows

"Low battery" if battery ≤ 20

"Normal" if battery between 21-80

"Full" if battery between > 80

But if phone is charging it should show "charging".
instead of any message.

```
Perc = int (input ("enter battery percentage:"))
```

```
is charging = int (input ("enter 1 if you are charging else 0:"))
```

```
if is charging == 1 :
```

```
    print ("charging")
```

```
elif perc  $\geq$  80:
```

```
    print ("Full")
```

```
elif perc  $>$  21 and perc  $<$  80:
```

```
    print ("Normal")
```

```
else :
```

```
    print ("low battery")
```

Output :-

Enter battery percentage = 80

enter 1 if you are charging else enter 0: 1

charging

5. Driving license check

A person gets a driving license if.

age ≥ 18 and passed driving test [1 = Yes]

But if age ≥ 60 , driving test is not required

Print ("eligible" or "Not eligible")

```
age = int (input ("enter your age:"))
```

```
istest = int (input ("enter 1 if driving test passed else 0: "))
```

```
if istest == 1:
```

```
    if age  $\geq 18$ :
```

```
        print ("eligible")
```

```
    else: print ("Not eligible")
```

```
elif age  $\geq 60$  and istest == 0:
```

```
    print ("eligible")
```

```
else:
```

```
    print ("Not eligible")
```

Output :-

enter your age : 68

enter 1 if during test passed else enter 0 : 0

eligible

6. Online food delivery

A restaurant gives free delivery if:

order amount ≥ 500 or user is a gold member

But if the distance is more than 10 km delivery is Never free

Print "free delivery" or "Delivery charged"

```
order = int(input("enter order Amount:"))
```

```
is gold = int(input("enter 1 if you're gold member else 0:"))
```

```
distance = int(input("enter distance:"))
```

```
if order  $\geq$  500:
```

```
    if distance  $\geq$  10:
```

```
        Print ("delivery charged")
```

```
    else:
```

```
        Print ("free delivery")
```

```
elif is gold == 1:
```

```
    if distance  $\geq$  10:
```

```
        Print ("delivery charged")
```

```
else:
```

```
    print ("delivery charged")
```

Output :-

enter order amount : 500

enter 1 if you're gold member else 0: 1

enter distance : 15

delivery charged.

7. Bank loan Approval

A bank approves a loan if :

Salary $\geq 30,000$ and credit score ≥ 700 OR

Salary $\geq 50,000$ [credit score [ignored]

Print "loan Approved" or "loan Rejected"

```
sal = int(input("Enter Salary :"))
```

```
score = int(input("Enter credit score :"))
```

```
if sal >= 30000 :
```

```
    if score >= 700 :
```

```
        print("loan Approved")
```

```
    elif sal >= 50000 :
```

```
        print("loan Approved")
```

```
    else :
```

```
        print("loan rejected")
```

```
else :
```

```
    print("loan rejected")
```

output :-

Enter Salary : 56000

Enter credit score : 0

loan approved

8 Electricity bill

units consumed:

first 100 units $\rightarrow$ 2/unit

Next 100 units $\rightarrow$ 3/unit

Above 200 units $\rightarrow$ 5/unit

Note : No loops

Print final bill Amount

```
units = int(input("enter no. of. units :"))
```

```
if units <= 100:
```

```
    bill = units * 2
```

```
elif units <= 200:
```

```
    bill = (100 * 2) + (unit - 100) * 3
```

```
else:
```

```
    bill = (100 * 2) + (100 * 3) + (unit - 200) * 5)
```

```
Print ("final bill Amount :", bill)
```

Output :-

Enter no. of. units : 800

Final bill Amount : 3500

9. Student Scholarship

A student gets scholarship if:

marks ≥ 85 And Family income < 500000

But if the student is a single parent child income condition is ignored

```
marks = int(input("enter marks:"))
```

```
income = int(input("enter income:"))
```

```
sparent = int(input("enter 1 if single parent else enter 0:"))
```

```
if marks  $\geq$  85:
```

```
    if income  $<$  500000:
```

```
        Print ("gets scholarship")
```

```
    elif sparent == 1:
```

```
        print ("gets scholarship")
```

```
else:
    print ("Not eligible for scholarship")
```

```
else:
    print ("Not eligible for scholarship")
```

Output :-

enter marks : 89

enter income : 596000

enter 1 if single parent else enter 0: 1

gets scholarship.

10. Online exam result

A student passes if :

theory ≥ 40 and practical ≥ 40

But if total ≥ 100 , pass even one is less than 40.

```
theory = int(input("enter theory marks :"))
```

```
practical = int(input("enter practical marks :"))
```

```
total = theory + practical
```

```
if theory  $\geq 40$  and practical  $\geq 40$ :
```

```
    print("pass")
```

```
elif total  $\geq 100$ :
```

```
    print("pass")
```

```
else:
```

```
    print("fail")
```

output :-

enter theory marks : 86

enter practical marks : 24

pass

11. Hotel room pricing

A hotel charges:

3000 per day for Normal days

4000 per day for weekend day

if customer stays more than 3 days, give 15% discount

Print final bill

```
nod = int(input("enter no of days stay:"))
```

```
is weekend = int(input("enter 1 if it is weekend else enter 0:"))
```

```
normal = 3000
```

```
weekends = 4000
```

```
bill = 1
```

```
if isweekend == 1:
```

```
    bill = nod * weekends
```

```
    print("final bill is :", bill)
```

```
else:
```

```
    bill = nod * normal.
```

```
    print("final bill is :", bill)
```

```
if nod >= 3:
```

```
    dis = bill * 0.15
```

```
    fin = bill - dis
```

```
    print("final bill... :", fin)
```

Output :-

enter no. of days . stay : 9

enter 1 if it weekend else enter 0: 0

final bill : 22950

12 Gaming level unlock

A game unlock next level if:

Score ≥ 100 OR player has a premium pass

But if player used cheating access is denied

```
Score = int (input ("enter score:"))
```

```
ispremium = int (input ("enter 1 if premium else enter 0:"))
```

```
ischeated = int (input ("enter 1 if cheated else enter 0:"))
```

```
if score  $\geq$  100 or ispremium == 1:
```

```
    if ischeated == 1:
```

```
        print ("Access denied")
```

```
    else:
```

```
        print ("game unlocks next level")
```

```
else:
```

```
    print ("Access denied")
```

Output :-

enter score : 12

enter 1 if premium else enter 0 : 1

enter 1 if cheated else enter 0 : 0

game unlocks next level

13. Mobile data usage.

A network gives unlimited data if:

daily usage ≤ 2 GB or user has unlimited plan

But if roaming is on unlimited data does not work

```
data = int(input("enter data used:"))
```

```
isunlimited = int(input("enter 1 if unlimited data else 0:"))
```

```
isroaming = int(input("enter 1 if roaming else 0:"))
```

```
if data <= 2 or isunlimited == 1:
```

```
    if isroaming == 1:
```

```
        Print("unlimited doesn't work")
```

```
    else:
```

```
        Print("unlimited data")
```

```
else:
```

```
    Print("No unlimited data")
```

Output :-

enter data used : 2

enter 1 if unlimited data else 0 : 0

enter 1 if roaming else 0 : 1

unlimited doesn't work

14. office entry system

An employee can enter the office if :

ID card is valid AND (fingerprint OR facescan matches)

But if it is a holiday, entry is denied for everyone

```
idvalid = int [input ("enter 1 if idvalid else enter 0:")]
```

```
fingerprint = int [input ("enter 1 if fingerprint valid else enter 0:")]
```

```
face scan = int [input ("enter 1 for facescan valid else 0:")]
```

```
is holiday = int [input ("enter 1 if holiday else 0:")]
```

```
if idvalid == 1 and (fingerprint == 1 or facescan == 1) :
```

```
    if isholiday == 1:
```

```
        Print ("Access denied a tdy is Holiday")
```

```
    else:
```

```
        print ("enter into office")
```

```
else:
```

```
    print ("access denied")
```

Output:-

enter 1 if idvalid else enter 0:1

enter 1 if fingerprint valid else enter 0:1

enter 1 if facescan valid else enter 0:0

enter 1 if holiday else enter 0:1.

access denied a tdy is Holiday.

15. Movie rating display

A movie app shows rating based on average score :

average $\geq 8.5 \rightarrow$ Excellent

average between 6.0 and 8.4 $\rightarrow$ Good

average $< 6.0 \rightarrow$ Average

But if the movie is marked as editor's choice always show recommended

Print the message

```
avgrate = float (input ("enter rating: "))
```

```
is editorchoice = int (input ("enter 1 if it is editor choice  
else enter 0: "))
```

```
if iseditorchoice == 1:
```

```
    Print ("recommended")
```

```
elif avgrate  $\geq$  8.5
```

```
    Print ("Excellent")
```

```
elif avgrate  $\geq$  6.0 and avgrate  $\leq$  8.4:
```

```
    Print ("good")
```

```
else: print ("average")
```

Output:-

enter rating : 7.5

enter 1 if it is editorchoice else enter 0:0

good

④ Discuss control statement and Decision making statements in python.

Introduction :-

In python programming, control statements are used to control the flow of execution of a program. Normally, Python executes statements sequentially control statements allows a program to make decisions, Repeat tasks or transfer control to another part of the program based on conditions

* Meaning and Importance of control statements

meaning :-

control statements determine which statement will be executed, how many times it be executed and under what condition it should be executed.

Importance :-

- enable decision making in programs
- Allow repetition of tasks using loops
- improve program efficiency and flexibility
- Helps in writing logical and dynamic programs
- essential for solving real world problems.

* Types of control statements in python

Python control statements are broadly classified into three types.

1. Decision - Making statements

used to execute different blocks of code based on conditions. They are

- * if
- * if - else
- * if - elif - else